

***Master IA2S - Graduate Programme***  
***IA Intelligence Artificielle, Science des données et***  
***Systèmes Cyber-Physiques***

**Module : Ingénierie logicielle pour la conception et le**  
**déploiement de systèmes d'IA**

**Compte rendu du TP1**

**Réalisé par :** DRIEF NISRINE

**Encadré par :** Mr CHIBANI

## **Objectif du tp :**

- Comprendre le rôle et les caractéristiques des trois formats de fichiers majeurs du Big Data : Avro, Parquet et ORC.
- Découvrir la différence entre un format orienté lignes (Avro) et un format orienté colonnes (Parquet, ORC).
- Comprendre la notion de schéma de données et son rôle dans la lecture d'un fichier binaire.
- Manipuler les bibliothèques Python dédiées : fastavro pour Avro, pandas + pyarrow pour Parquet, pyarrow.orc pour ORC.
- Créer un petit jeu de données en Python pour chaque format (liste de dictionnaires, DataFrame, table Arrow).
- Enregistrer des données dans chacun des trois formats binaires.
- Relire un fichier binaire et afficher son contenu de manière lisible.
- Consulter le schéma d'un fichier sans avoir à lire toutes les données.
- Sélectionner uniquement certaines colonnes lors de la lecture (possible en Parquet et ORC, impossible en Avro).
- Appliquer des filtres et des agrégations simples sur les données (moyenne, somme par catégorie, comptage).
- Comparer les caractéristiques des trois formats (organisation, schéma, compression, usage, écosystème).

## **Les formats AVRO, Parquets, ORC :**

### ➤ Avro

Avro est un format binaire orienté lignes. Les données sont stockées enregistrement par enregistrement, comme dans un carnet où chaque page correspond à une ligne. Son atout principal est que le schéma est embarqué dans le fichier : la description des champs (noms, types) est écrite dans l'en-tête, ce qui permet à n'importe quel programme de décoder les données sans avoir besoin de connaître le schéma à l'avance.

Avro supporte l'évolution de schéma : on peut ajouter un champ optionnel sans casser les applications existantes. C'est pour cette raison qu'il est très utilisé dans les systèmes de streaming (Apache Kafka) et dans les couches Bronze des architectures Médaille, où on stocke les données brutes telles qu'elles arrivent.

### ➤ Parquet

Parquet est un format binaire orienté colonnes. Contrairement à Avro, les valeurs d'une même colonne sont stockées ensemble. Ce mode de stockage présente plusieurs avantages :

- Lecture sélective : on ne lit que les colonnes nécessaires à la requête (projection pushdown).
- Compression efficace : les valeurs d'une même colonne se ressemblent et se compressent mieux.

- Statistiques internes : Parquet stocke les valeurs min/max par bloc, ce qui permet de sauter des blocs entiers sans les lire (`predicate pushdown`).

Parquet est devenu le standard de l'analyse de données : il est lu par Spark, Pandas, DuckDB, Trino, BigQuery, Athena, et il sert de base aux formats de table modernes (Iceberg, Delta Lake). On l'utilise dans les couches Silver et Gold du Lakehouse.

### ➤ ORC

ORC (Optimized Row Columnar) est également un format binaire orienté colonnes, très proche de Parquet dans son fonctionnement. Il a été créé à l'origine pour l'écosystème Hadoop / Hive, où il est encore très utilisé. ORC offre une compression performante et des statistiques internes (min/max, nombre de valeurs) qui accélèrent les requêtes analytiques.

En pratique, Parquet est aujourd'hui plus universel, mais ORC reste présent dans les entrepôts de données Hadoop et dans certains systèmes hérités.

## Création d'un petit jeu de données en Python

Pour chaque format, nous avons construit un petit jeu de données en Python :

Jeu 1 : Relevés de capteurs (Avro)

Quatre relevés de capteurs IoT, avec les champs : identifiant, zone, température, humidité, actif. Ce jeu a été choisi car il représente un cas typique d'ingestion de données en streaming.

Jeu 2 : Ventes (Parquet)

Cinq ventes de produits informatiques, avec les champs : identifiant, produit, catégorie, quantité, montant en euros. Ce jeu représente un cas typique d'analyse commerciale.

Jeu 3 : Observations d'espèces (ORC)

Cinq observations naturalistes, avec les champs : identifiant, site, espèce, score de confiance, validée. Ce jeu représente un cas typique de classification automatique.

Les données ont été créées directement en Python, soit sous forme de liste de dictionnaires (Avro), soit sous forme de `DataFrame` Pandas (Parquet), soit sous forme de table Arrow (ORC).

## Enregistrement des données dans chacun de ces formats

### ➤ Écriture d'un fichier Avro

Pour écrire en Avro, nous avons utilisé la bibliothèque `fastavro`. Le schéma a été défini sous forme de dictionnaire Python, puis converti en schéma Avro valide avec `parse_schema`. L'écriture se fait avec `writer`, en ouvrant le fichier en mode **binaire** ("`wb`")

Le mode binaire est **obligatoire** car Avro est un format binaire : en mode texte, Python corromprait les données.

### ➤ Écriture d'un fichier Parquet

Pour écrire en Parquet, nous avons utilisé Pandas avec le moteur PyArrow

Le paramètre `index=False` évite d'écrire l'index Pandas (0, 1, 2...) dans le fichier, ce qui ajouterait une colonne inutile.

### ➤ Écriture d'un fichier ORC

Pour écrire en ORC, nous avons utilisé PyArrow, qui manipule des tables Arrow

PyArrow gère automatiquement l'ouverture en mode binaire et l'écriture des métadonnées.

## Lecture et affichage du contenu d'un fichier

### ➤ Relecture d'un fichier Avro

Le fichier est ouvert en **binaire** ("rb"). Le `reader` lit l'en-tête, récupère le schéma via `writer_schema`, puis parcourt les enregistrements. La conversion en liste (`list(lecteur)`) charge tous les enregistrements en mémoire.

### ➤ Relecture d'un fichier Parquet

On charge tout le fichier dans un `DataFrame` Pandas, ce qui permet ensuite d'utiliser toutes les méthodes Pandas (filtrage, groupby, etc.). L'affichage avec `to_string(index=False)` évite d'afficher l'index.

### ➤ Relecture d'un fichier ORC

On lit toute la table ORC en mémoire, puis on la convertit en `DataFrame` Pandas pour bénéficier des mêmes méthodes que pour Parquet.



## Consultation du schéma et sélection de données

- **Avro** : le schéma est accessible via `lecteur.writer_schema`. Il est **stocké dans le fichier** (schéma embarqué).

```
(.venv) nisirine@DESKTOP-FN8QJPT:~/tp_formats_big_data$ python exercice_avro.py
Schéma Avro :
{'type': 'record', 'name': 'fr.upec.iot.ReleveCapteur', 'fields': [{'name': 'capteur_id', 'type': 'string'}, {'name': 'zone', 'type': 'string'}, {'name': 'temperature', 'type': 'double'}, {'name': 'humidite', 'type': 'int'}, {'name': 'actif', 'type': 'boolean'}]}

Enregistrements :
{'capteur_id': 'C001', 'zone': 'Laboratoire', 'temperature': 21.8, 'humidite': 45, 'actif': True}
{'capteur_id': 'C002', 'zone': 'Couloir', 'temperature': 23.4, 'humidite': 41, 'actif': True}
{'capteur_id': 'C003', 'zone': 'Salle serveur', 'temperature': 19.7, 'humidite': 38, 'actif': True}
{'capteur_id': 'C004', 'zone': 'Atelier', 'temperature': 24.1, 'humidite': 52, 'actif': False}

Relevés avec température > 22 °C :
{'capteur_id': 'C002', 'zone': 'Couloir', 'temperature': 23.4, 'humidite': 41, 'actif': True}
{'capteur_id': 'C004', 'zone': 'Atelier', 'temperature': 24.1, 'humidite': 52, 'actif': False}

Température moyenne : 22.25 °C
(.venv) nisirine@DESKTOP-FN8QJPT:~/tp_formats_big_data$ ls -l data/
total 4
-rw-r--r-- 1 nisirine nisirine 436 Sep 13 12:23 capteurs.avro
(.venv) nisirine@DESKTOP-FN8QJPT:~/tp_formats_big_data$ python exercice_parquet.py
```

- **Parquet** : le schéma est accessible via `pq.ParquetFile(fichier).schema_arrow`, sans lire les données

```
(.venv) nisirine@DESKTOP-FN8QJPT:~/tp_formats_big_data$ python exercice_parquet.py
Schéma Parquet :
vente_id: int64
produit: large_string
categorie: large_string
quantite: int64
montant_eur: double
-- schema metadata --
pandas: '{"index_columns": [], "column_indexes": [], "columns": [{"name": ' + 666

Toutes les ventes :
vente_id produit categorie quantite montant_eur
1 Clavier Accessoire 2 79.8
2 Écran Affichage 1 249.9
3 Souris Accessoire 3 74.7
4 Écran Affichage 2 499.8
5 Casque Audio 1 89.9

Colonnes sélectionnées :
produit montant_eur
Clavier 79.8
Écran 249.9
Souris 74.7
Écran 499.8
Casque 89.9

Ventes >= 100 € :
vente_id produit categorie quantite montant_eur
2 Écran Affichage 1 249.9
4 Écran Affichage 2 499.8

Chiffre d'affaires par catégorie :
categorie
Accessoire 154.5
Affichage 749.7
Audio 89.9
Name: montant_eur, dtype: float64
(.venv) nisirine@DESKTOP-FN8QJPT:~/tp_formats_big_data$ ls -l data/
total 8
```

- **ORC** : le schéma est accessible via `orc.ORCFile(fichier).schema`.

```
(.venv) nrisine@DESKTOP-FH8QJPT:~/tp_formats_big_data$ python exercice_orc.py
Schéma ORC :
observation_id: int64
site: string
espece: string
confiance: double
validee: bool

Toutes les observations :

```

| observation_id | site | espece               | confiance | validee |
|----------------|------|----------------------|-----------|---------|
| 101            | CDG  | Faucon crécerelle    | 0.96      | True    |
| 102            | CDG  | Renard roux          | 0.84      | False   |
| 103            | ZAG  | Mésange charbonnière | 0.93      | True    |
| 104            | CLUJ | Lièvre               | 0.78      | False   |
| 105            | ZAG  | Hirondelle           | 0.91      | True    |

```

Colonnes sélectionnées :

```

| espece               | confiance |
|----------------------|-----------|
| Faucon crécerelle    | 0.96      |
| Renard roux          | 0.84      |
| Mésange charbonnière | 0.93      |
| Lièvre               | 0.78      |
| Hirondelle           | 0.91      |

```

Observations avec une confiance >= 0.90 :

```

| observation_id | site | espece               | confiance | validee |
|----------------|------|----------------------|-----------|---------|
| 101            | CDG  | Faucon crécerelle    | 0.96      | True    |
| 103            | ZAG  | Mésange charbonnière | 0.93      | True    |
| 105            | ZAG  | Hirondelle           | 0.91      | True    |

```

Observations par site :

```

| site | count |
|------|-------|
| CDG  | 2     |
| ZAG  | 2     |
| CLUJ | 1     |

```

Name: count, dtype: int64

```

Dans les trois cas, le schéma décrit les **noms des colonnes** et leurs **types** (chaîne, entier, décimal, booléen...).

➤ Sélection de colonnes

Parquet et ORC permettent de **ne lire que certaines colonnes**

Cette fonctionnalité est **impossible en Avro**, qui est orienté lignes : pour accéder à une colonne, il faut lire tous les enregistrements.

➤ Filtrage et agrégation

Nous avons appliqué les mêmes opérations sur les trois jeux de données :

- **Filtrage** : relevés avec température > 22 °C (Avro), ventes ≥ 100 € (Parquet), observations avec confiance ≥ 0,90 (ORC).
- **Agrégation** : température moyenne (Avro), chiffre d'affaires par catégorie (Parquet), nombre d'observations par site (ORC).

Ces opérations ont été réalisées avec des filtres booléens (Pandas) ou des boucles (Avro).



## Comparaison des caractéristiques des trois formats

| Critère                            | Avro                                   | Parquet                                    | ORC                         |
|------------------------------------|----------------------------------------|--------------------------------------------|-----------------------------|
| <b>Organisation principale</b>     | Orienté lignes                         | Orienté colonnes                           | Orienté colonnes            |
| <b>Schéma inclus</b>               | Oui (JSON dans l'en-tête)              | Oui (métadonnées)                          | Oui (métadonnées)           |
| <b>Lecture de colonnes ciblées</b> | Non                                    | Oui                                        | Oui                         |
| <b>Compression</b>                 | Bonne (Snappy, Deflate)                | Excellente (Snappy, ZSTD)                  | Excellente (Zlib, Snappy)   |
| <b>Statistiques internes</b>       | Limitées                               | Oui (min/max par bloc)                     | Oui (min/max par bloc)      |
| <b>Évolution de schéma</b>         | Native                                 | Partielle                                  | Partielle                   |
| <b>Support Schema Registry</b>     | Oui (natif)                            | Non                                        | Non                         |
| <b>Usage principal</b>             | Streaming, échange de messages, Bronze | Analytique, Big Data, Silver/Gold          | Analytique, écosystème Hive |
| <b>Bibliothèque Python</b>         | <code>fastavro</code>                  | <code>pyarrow</code> , <code>pandas</code> | <code>pyarrow.orc</code>    |
| <b>Moteurs compatibles</b>         | Kafka, Spark, Flink                    | Spark, DuckDB, Trino, BigQuery, Pandas     | Hive, Spark, Impala         |

## Taille des fichiers produits

```
(.venv) nistrine@DESKTOP-FN8QUP:~/tp_formats_big_data$ ls -l data/
total 12
-rw-r--r-- 1 nistrine nistrine  436 Sep 13 12:23 capteurs.avro
-rw-r--r-- 1 nistrine nistrine  904 Sep 13 12:25 observations.orc
-rw-r--r-- 1 nistrine nistrine 3542 Sep 13 12:24 ventes.parquet
(.venv) nistrine@DESKTOP-FN8QUP:~/tp_formats_big_data$ cd ~
```

Avro et Parquet ne s'opposent pas : ils répondent à des besoins différents.

- Avro excelle pour écrire vite des événements, et son schéma embarqué le rend idéal pour le streaming.
- Parquet et ORC excellent pour analyser de gros volumes, grâce à leur stockage colonnaire, leur compression et leurs statistiques.
- ORC est historiquement lié à l'écosystème Hadoop / Hive, tandis que Parquet est devenu le standard universel.

## Synthèse :

- Ce TP a permis de manipuler concrètement trois formats de fichiers essentiels du monde de la donnée : Avro, Parquet et ORC.
- Nous avons appris à écrire et relire des données dans chacun de ces formats avec Python.
- Les bibliothèques utilisées (fastavro, pandas, pyarrow) offrent une API simple et cohérente pour ces opérations.
- Avro s'est révélé particulièrement adapté à l'écriture rapide et à l'échange de messages : son schéma embarqué et son orientation lignes en font le format idéal pour le streaming et la couche Bronze.
- Parquet s'est distingué par sa rapidité de lecture sur des colonnes ciblées et sa compression efficace : il est idéal pour l'analyse et les couches Silver et Gold.
- ORC est apparu comme un cousin de Parquet, avec les mêmes forces, mais historiquement lié à l'écosystème Hadoop / Hive.
- La comparaison des tailles sur un petit jeu de données n'est pas représentative : les métadonnées dominent. C'est à grande échelle que ces formats révèlent leurs avantages.
- Ces trois formats ne sont pas concurrents mais complémentaires : le choix dépend de l'usage (écrire vite vs lire efficacement).